SKT SOFTWARE SOLUTION	**EMPRESA DE DESARROLLO E INNOVACIÓN**	**CÓDIGO:** MI-DES-ESP-001
	PROCESO: DESARROLLO DE SOFTWARE	**VERSIÓN:** 1.0
	Desarrollo del Sistema LUPSI	**ELABORACIÓN:** 25/03/2026
	Especificación de Arquitectura REST y Contratos de API	**ÚLTIMA REVISIÓN:** 25/03/2026

1. Título

Definición de Arquitectura REST y Contratos de API del Sistema Web Progresivo (PWA) de Gestión Médica LUPSI: Módulos de Identidad, Agendamiento Clínico y Control de Acceso Basado en Roles

2. Introducción

El presente documento especifica formalmente la arquitectura orientada a servicios REST del backend del proyecto PWA Gestión Médica LUPSI, desarrollado para el Centro de Especialidades Médicas y Psicológicas LUPSI con sede en la ciudad de Ambato, Ecuador.

La solución propuesta migra un modelo de gestión 100 % manual — basado en WhatsApp, papel e intervención directa de una Psicóloga Clínica como recepcionista — hacia una plataforma de autoservicio digital de *fricción cero* (*Zero Friction Self-Service*). El sistema automatiza el ciclo completo de agendamiento médico, valida algorítmicamente la identidad de los pacientes mediante el **Algoritmo Módulo 10** de la cédula ecuatoriana, y garantiza el aislamiento de los datos clínicos sensibles mediante políticas de seguridad a nivel de base de datos.

El backend está construido sobre **NestJS** con TypeScript estricto, conectado a una base de datos **PostgreSQL administrada en Supabase**, aplicando **Row Level Security (RLS)** y **Control de Acceso Basado en Roles (RBAC)** para garantizar que los datos médicos sean inaccesibles al personal administrativo, incluso si la solicitud proviene de actores autenticados dentro del mismo sistema.

Este documento sirve como contrato técnico vinculante entre el equipo de backend (Daniel Luisa y Sebastián Ortiz) y el equipo de frontend (Alex Guachi), asegurando que ambas capas operen sobre acuerdos explícitos de interfaz, formato de datos y reglas de seguridad, alineados con el cronograma de 73 días laborables definido en la planificación del proyecto.

3. Objetivo

Especificar formalmente los contratos de interfaz REST (endpoints, métodos HTTP, DTOs de solicitud/respuesta, códigos de estado y requisitos de autorización) que expone el backend NestJS del sistema LUPSI, con el fin de garantizar una integración coherente, segura y predecible con la capa de presentación Angular PWA

y cumplir con los requerimientos funcionales y de seguridad definidos en el Sprint Backlog del proyecto.

4. Alcance

Este documento comprende:

- Descripción de la arquitectura de módulos del backend NestJS.
- Especificación de los endpoints REST organizados por módulo funcional.
- Definición de los roles de acceso (RBAC) y niveles de autorización por endpoint.
- Descripción de los contratos de datos de entrada (DTOs de solicitud).
- Descripción de los contratos de datos de salida (DTOs de respuesta).
- Códigos de respuesta HTTP aplicables y descripción de errores estandarizados.
- Consideraciones de seguridad y restricciones de acceso para datos clínicos sensibles.

Quedan fuera del alcance de este documento:

- El diseño de vistas y componentes Angular (documentado por separado en el documento de Wireframes y Navegación).
- La configuración de infraestructura DevOps y despliegue en Render.

5. Definiciones

API REST: Interfaz de programación de aplicaciones que sigue los principios de la arquitectura representacional de transferencia de estado (REST), usando HTTP como protocolo de comunicación.

NestJS: Framework de Node.js basado en TypeScript que implementa patrones de diseño orientados a objetos y programación modular para construir backends escalables.

DTO (Data Transfer Object): Objeto de datos que define la estructura exacta de los datos que viajan entre capas del sistema (solicitud/respuesta HTTP).

JWT (JSON Web Token): Token firmado criptográficamente que Supabase Auth emite al autenticarse. Contiene el identificador único del usuario (`auth.uid()`) y los *Custom Claims* de rol.

RBAC (Role-Based Access Control): Modelo de control de acceso donde los permisos se otorgan en función del rol asignado al usuario (PATIENT, DOCTOR, RECEPTIONIST, ADMIN).

RLS (Row Level Security): Característica de PostgreSQL que aplica políticas de filtrado a nivel de fila de tabla basadas en la identidad del usuario JWT activo.

Guard (NestJS): Clase que implementa `CanActivate` e intercepta la ejecución de un controlador para verificar autorización antes de procesar la solicitud.

Módulo 10: Algoritmo matemático utilizado por el Registro Civil del Ecuador para validar la veracidad de los 10 dígitos de la cédula de identidad de personas naturales.

Slot de disponibilidad: Bloque de tiempo libre en la agenda de un doctor, sin cita activa (`status = 'SCHEDULED'`), calculado dinámicamente por el motor de agendamiento.

Endpoint: URL específica expuesta por la API REST a la cual los clientes envían solicitudes HTTP.

6. Arquitectura de Módulos NestJS

6.1. Diagrama de Organización de Módulos

El backend se organiza en módulos de dominio siguiendo el principio *Single Responsibility* de SOLID y el patrón de *Feature Modules* de NestJS. Cada módulo encapsula su propio controlador, servicio(s) y acceso a datos, y se conecta al módulo raíz `AppModule`.

Módulo	Responsabilidad Principal	Dev
`IamModule`	Validación de cédula (Módulo 10), Guards JWT y RBAC, estrategias de autenticación con Supabase Auth.	Daniel L.
`AuthModule`	Endpoints de registro, login y logout. Orquesta la comunicación con `supabase.auth.*` y la escritura en `public.profiles`.	Daniel L.
`ProfilesModule`	CRUD de perfiles de pacientes y doctores (datos no clínicos). Consume `public.patients` y `public.doctors`.	Daniel L. & Sebas. O.
`SchedulingModule`	Motor de agendamiento concurrente. Gestión de disponibilidad, creación y cancelación de citas. Aplica restricción GiST.	Sebas. O. & Daniel L.
`ClinicalModule`	Gestión de historias clínicas, subida de PDFs a Cloudinary y lectura restringida por rol `DOCTOR/PATIENT`.	Daniel L. & Sebas. O.
`StorageModule`	Integración con Cloudinary SDK. Recibe archivos, sube el binario y retorna la URL estática al módulo solicitante.	Daniel L.

Tabla 1: Organización de módulos funcionales del backend LUPSI.

6.2. Convenciones Globales de la API

- **Base URL Local:** `http://localhost:3000`
- **Base URL Producción:** `https://lupsi-api.onrender.com`
- **Prefijo global:** `/api/v1`
- **Formato de datos:** JSON (`Content-Type: application/json`)
- **Autenticación:** Header `Authorization: Bearer {jwt_token}`
- **Zona Horaria:** Todas las fechas se manejan en formato **ISO 8601 UTC** (`TIMESTAMP WITH TIME ZONE`).
- **Idioma de errores:** Español. Mensajes descriptivos orientados al equipo frontend para traducción a la UI.

7. Especificación de Endpoints por Módulo

7.1. Módulo de Autenticación — `/api/v1/auth`

Método	Ruta	Descripción	Acceso / Código
POST	`/register`	Registra un nuevo usuario. Valida el DNI con Módulo 10, crea usuario en Supabase Auth e inserta en `public.profiles` y `public.patients`.	Público — **201**
POST	`/login`	Autentica un usuario existente. Retorna JWT de Supabase con Custom Claims de rol.	Público — **200**
POST	`/logout`	Invalida la sesión activa del usuario en Supabase Auth.	JWT válido — **200**

Tabla 2: Endpoints del módulo de autenticación.

7.1.1. POST `/auth/register` — Contrato de Datos

Request Body (DTO):

Campo	Tipo	Requerido	Descripción / Validación
`email`	string	Sí	Email válido. Será el identificador primario de Supabase Auth.
`password`	string	Sí	Mínimo 8 caracteres, al menos 1 mayúscula y 1 número.
`dni`	string	Sí	10 dígitos numéricos. Validado con Algoritmo Módulo 10 antes de cualquier escritura en DB.
`first_name`	string	Sí	Máximo 100 caracteres.
`last_name`	string	Sí	Máximo 100 caracteres.
`phone`	string	No	Formato libre. Recomendado E.164 (`+593...`).
`date_of_birth`	string	No	Fecha en formato `YYYY-MM-DD`.

Tabla 3: DTO de solicitud para registro de paciente.

Response Body (201 Created):

```
{
  "message": "Registro exitoso. Verifica tu correo electrónico.",
  "userId": "uuid-del-usuario"
}
```

Errores posibles:

Código HTTP	Mensaje de error
400	`El DNI es matemáticamente inconsistente (Algoritmo Módulo 10).`
400	`El DNI no pertenece a una persona natural.`
409	`El correo electrónico ya está registrado en el sistema.`
409	`El DNI ya se encuentra asociado a un perfil existente.`

Tabla 4: Códigos de error del endpoint de registro.

7.2. Módulo de Perfiles — `/api/v1/profiles`

Método	Ruta	Descripción	Acceso / Código
GET	`/me`	Retorna los datos del perfil del usuario autenticado actualmente.	Cualquier rol — **200**
PATCH	`/me`	Actualiza datos del perfil propio (teléfono, fecha de nacimiento).	PATIENT — **200**
GET	`/patients`	Lista todos los pacientes registrados.	RECEPTIONIST, DOCTOR — **200**
GET	`/patients/:id`	Retorna el perfil de un paciente específico por su UUID.	RECEPTIONIST, DOCTOR — **200**
GET	`/doctors`	Lista todos los doctores disponibles con su especialidad.	Autenticado — **200**

Tabla 5: Endpoints del módulo de perfiles.

7.3. Módulo de Agendamiento — `/api/v1/appointments`

Método	Ruta	Descripción	Acceso / Código
GET	`/slots`	Retorna horarios disponibles de un doctor en una fecha. Parámetros: `doctorId`, `date`.	PATIENT — **200**
POST	`/`	Crea una nueva cita. El backend calcula `appointment_end_time` y ejecuta la restricción GiST anti-solapamiento.	PATIENT — **201**
GET	`/my`	Lista todas las citas del paciente autenticado, ordenadas por fecha.	PATIENT — **200**
GET	`/daily`	Retorna las citas del día al panel de la recepcionista (sin datos clínicos: nombre, hora, estado).	RECEPTIONIST — **200**
PATCH	`/:id/cancel`	Cancela una cita. Cambia `status` a `CANCELLED` y libera el slot automáticamente.	PATIENT (propio), RECEPTIONIST — **200**
PATCH	`/:id/complete`	Marca la cita como `COMPLETED`. Solo el doctor que la atiende puede ejecutarlo.	DOCTOR — **200**

Tabla 6: Endpoints del módulo de agendamiento (motor concurrente).

7.3.1. POST `/appointments` — Contrato de Datos

Request Body (DTO):

Campo	Tipo	Requerido	Descripción
`doctor_id`	string (UUID)	Sí	ID del doctor seleccionado.
`appointment_time`	string (ISO 8601)	Sí	Fecha y hora de inicio de la cita. Ej: `2026-04-01T10:00:00Z`.
`duration_minutes`	number	No	Duración en minutos. Default: 45. El backend calcula `appointment_end_time`.

Tabla 7: DTO de solicitud para creación de cita médica.

Lógica de concurrencia: Si dos solicitudes simultáneas intentan agendar al mismo doctor en el mismo rango horario, PostgreSQL devolverá una excepción de exclusión GiST y el backend retornará `HTTP 409 Conflict` con el mensaje: *"El horario seleccionado ya no está disponible. Por favor elige otro slot."* La primera solicitud que haya comprometido la transacción ganará el slot.

7.4. Módulo Clínico — `/api/v1/clinical`

Método	Ruta	Descripción	Acceso / Código
POST	`/records`	Crea una nueva historia clínica con diagnóstico y sube el PDF de indicaciones a Cloudinary. Almacena solo la URL en PostgreSQL.	DOCTOR — **201**
GET	`/records/patient/:id`	Lista todas las historias clínicas de un paciente específico.	DOCTOR, PATIENT (propio) — **200**
GET	`/records/:id`	Retorna los detalles de una historia clínica, incluyendo la URL del PDF en Cloudinary.	DOCTOR, PATIENT (propio) — **200**

Tabla 8: Endpoints del módulo clínico (acceso restringido, RECEPTIONIST bloqueada).

Nota de Seguridad Crítica: El rol `RECEPTIONIST` no tiene ninguna política RLS ni Guard que le permita acceder a estos endpoints. Una solicitud con JWT de recepcionista recibirá `HTTP 403 Forbidden`: *"Acceso denegado. No tiene permisos para visualizar información clínica."* Este bloqueo opera en doble capa: Guard NestJS (capa de aplicación) y política RLS de PostgreSQL (capa de datos).

8. Modelo de Respuesta de Error Estándar

Todos los errores del backend siguen el siguiente formato JSON unificado para facilitar el manejo en el frontend Angular:

```
{
  "statusCode": 400,
  "error": "Bad Request",
  "message": "El DNI es matemáticamente inconsistente (Algoritmo Módulo 10).",
  "timestamp": "2026-04-01T10:00:00.000Z",
  "path": "/api/v1/auth/register"
}
```

9. Responsabilidades y Recursos

9.1. Responsabilidades

Rol	Responsabilidades
Angel Ayuquina – Gestor del Proyecto	Aprobación del contrato de API, seguimiento de cumplimiento durante los Sprints y gestión de cambios de alcance que afecten los endpoints definidos.
Sebastián Ortiz – Arquitectura BD y Cloud	Diseño del esquema de tablas PostgreSQL, configuración de Supabase, definición de políticas RLS y revisión técnica de los DTOs que interactúan con la base de datos.
Daniel Luisa – Backend y QA	Implementación de todos los controladores y servicios NestJS, codificación de Guards RBAC, pruebas unitarias e integración de la API con Supabase Auth y Cloudinary.
Alex Guachi – Frontend y UX/UI	Consumo de todos los endpoints especificados en este documento desde Angular. Responsable de reportar inconsistencias entre el contrato y el comportamiento real de la API.

9.2. Recursos

Categoría	Detalle
Humanos	Equipo de 4 desarrolladores organizados bajo metodología Scrum híbrida.
Tecnológicos	▪ NestJS 10 + TypeScript 5 (Backend). ▪ Angular 21 (Frontend PWA). ▪ Supabase (PostgreSQL + Auth + RLS). ▪ Cloudinary (Storage de PDFs clínicos). ▪ Docker + Render (Despliegue en producción). ▪ Postman (Testing manual y PenTesting de endpoints).
Normativos	▪ ISO/IEC 25010 (Calidad de producto software). ▪ ISO/IEC 27001 (Seguridad de la información). ▪ OWASP API Security Top 10 2023. ▪ Ley Orgánica de Protección de Datos Personales (LOPDP) — Ecuador.
Presupuesto	Infraestructura acotada a $150 USD. Los servicios Supabase Free Tier y Render Free Tier cubren el tráfico estimado del MVP.

10. Normativa Legal y Técnica Aplicable

10.1. OWASP API Security Top 10 (2023)

El diseño de cada endpoint fue evaluado contra el listado OWASP API Security Top 10 en sus categorías más críticas para este sistema:

- **API1 — Broken Object Level Authorization:** Mitigado mediante Guards

RBAC en NestJS y políticas RLS en PostgreSQL (doble barrera). Un paciente jamás puede solicitar el perfil clínico de otro paciente.

- **API2 — Broken Authentication:** Mitigado delegando completamente la emisión y verificación de JWT a Supabase Auth, que implementa estándares modernos de seguridad sin dependencia de código propio.
- **API3 — Broken Object Property Level Authorization:** Mitigado mediante DTOs de respuesta estrictos que nunca exponen campos internos de la base de datos (como `auth.uid()` crudo o el hash de contraseña).
- **API8 — Security Misconfiguration:** Mitigado mediante la separación de `SUPABASE_ANON_KEY` (frontend) y `SUPABASE_SERVICE_ROLE_KEY` (backend exclusivo), almacenadas como variables de entorno y nunca en repositorio.

10.2. Ley Orgánica de Protección de Datos Personales — LOPDP (Ecuador)

Los artículos 7 y 19 de la LOPDP exigen que los datos de salud sean tratados como datos sensibles de categoría especial, requiriendo consentimiento explícito del titular y controles técnicos de acceso. Este sistema implementa cumplimiento mediante:

- Aceptación explícita de términos en el flujo de registro (`POST /auth/register`).
- Aislamiento técnico de las tablas `medical_records` mediante RLS que bloquea al rol `RECEPTIONIST` incluso con credenciales válidas.
- Almacenamiento de PDFs clínicos en Cloudinary (fuera de la base de datos) con URLs de acceso privado no indexables.

11. Método / Políticas

11.1. Política de Versionado de la API

- La API usa versionado en la URL (`/api/v1/`). Cualquier cambio que rompa la compatibilidad hacia atrás requerirá incrementar a `/api/v2/` y ser aprobado por el Gestor del Proyecto.

- Los cambios aditivos (nuevos campos opcionales en respuesta) no requieren cambio de versión.
- Cualquier modificación en los contratos de este documento debe registrarse en la sección de Control de Cambios.

11.2. Política de Testing de Endpoints

- Antes de cada Pull Request al repositorio `lupsi-api-backend`, los endpoints afectados deben tener al menos una prueba unitaria del servicio y una prueba de integración del controlador.
- El módulo Clínico y los Guards RBAC son objetivo de PenTesting manual con Postman en el Sprint 4, simulando tokens falsificados con roles incorrectos.

12. Firmas de Responsabilidad

Rol	Nombre / Representante	Firma (QR)	Fecha
Gestor del Proyecto	Angel Ayuquina		25/03/2026
Arquitecto BD y Cloud	Sebastián Ortiz		25/03/2026
Desarrollador Backend y QA	Daniel Luisa		25/03/2026
Desarrollador Frontend y UX/UI	Alex Huachi		25/03/2026

13. Control de Cambios

Versión	Fecha	Descripción del Cambio	Responsable
1.0	25/03/2026	Creación inicial del documento de Arquitectura y APIs. Definición de los 4 módulos principales y sus contratos de datos.	Sebastián Ortiz

14. Anexos

A. Codificación del Documento

A.1 Referencia de Codificación

Campo	Valor
Código documento	`MI-DES-ESP-001`
Título	Especificación de Arquitectura REST y Contratos de API
Versión	1.0
Fecha emisión	2026-03-25
Autor / Responsable	Sebastián Ortiz (Arquitectura BD y Cloud)
Motivo de emisión	Definición inicial del contrato de integración Backend/Frontend
Aprobación	Angel Ayuquina / 25-03-2026

B. Resumen de Roles y Permisos por Endpoint

Endpoint	PATIENT	DOCTOR	RECEPTIONIS	ADMIN
`POST /auth/register`	✓	—	—	—
`POST /auth/login`	✓	✓	✓	✓
`GET /profiles/me`	✓	✓	✓	✓
`GET /profiles/patients`	—	✓	✓	✓
`GET /profiles/doctors`	✓	✓	✓	✓
`GET /appointments/slots`	✓	—	—	✓
`POST /appointments`	✓	—	—	✓
`GET /appointments/my`	✓	✓	—	✓
`GET /appointments/daily`	—	—	✓	✓
`PATCH /:id/cancel`	✓ (propio)	—	✓	✓
`PATCH /:id/complete`	—	✓ (propio)	—	✓
`POST /clinical/records`	—	✓	**BLOQUEADO**	✓
`GET /clinical/records/...`	✓ (propio)	✓	**BLOQUEADO**	✓

Tabla B.1: Matriz de acceso por rol (RBAC). **BLOQUEADO** = HTTP 403 en Guard NestJS + RLS PostgreSQL.